

CURSO DE PYTHON

Compactação com Huffman

Aula extra: da frequência à árvore, do bit ao arquivo .huff

Python 3 · VS Code · hash + árvore + prioridade

APOSTILA ELABORADA POR

Professor Celso Brunno Rocha Custódio de Campos

Como esta apostila funciona

Um encontro de 2h30. Primeiro a mensagem BANANA no quadro (contar, juntar, árvore, bits). Depois o código em exemplos/huffman.py e um recado honesto sobre o que o ZIP faz a mais. Não substitui as 8 aulas do curso: usa o que elas já ensinaram.

Uso desta apostila

© 2026 Celso Brunno Rocha Custódio de Campos. Material de uso didático.

É vedada a reprodução, distribuição ou comercialização sem autorização do autor.

O aluno pode usar o conteúdo para estudar e montar o projeto do curso.

Sumário

Clique no título para ir à seção.

Antes de começar	3
O que você vai construir hoje	3
O que precisa já saber	3
Roteiro das 2h30	3
AULA EXTRA	
Compactação com Huffman	5
O problema (8 bits para tudo)	5
Resposta em uma frase	5
Código prefixo-livre	5
Exemplo na mão: BANANA	6
Juntar sempre os dois menores	6
Ler os bits na árvore	7
Compactar letra a letra	8
Descompactar bit a bit	8
Live coding	9
O arquivo .huff	12
O que o ZIP faz a mais	13
Prática de hoje	13

Antes de começar

Esta apostila é de *um* encontro (2h30). O alvo não é o ZIP completo: é a ideia que faz o ZIP funcionar — gastar poucos bits no que aparece muito.

Objetivo desta aula (2h30)

Sair da aula sabendo compactar e descompactar **BANANA** na mão, explicar por que o código precisa ser prefixo-livre, e rodar `exemplos/huffman.py`.

O que você vai construir hoje

- ▶ Uma tabela de frequência (o mesmo espírito da Aula 4 — contar por chave).
- ▶ Uma floresta de árvores que vai juntando os dois menores até sobrar uma.
- ▶ Uma árvore de Huffman: folha = letra, ramo esquerdo = 0, direito = 1.
- ▶ Um arquivo `.huff` com a tabela e os bits, e a volta ao texto original.

O que precisa já saber

Não é a primeira aula do curso. Você já viu nó com `esquerda/direita` (Aula 5) e a ideia de “sempre o menor na frente” (Aula 7). Hoje a árvore *não* é BST: não comparamos letra com letra para decidir o lado. O lado é só o bit.

- ▶ Classe com atributos, `if` e `for`.
- ▶ Dicionário: `freq[letra] = freq.get(letra, 0) + 1`.
- ▶ Árvore: um nó, dois filhos, folha é o nó sem (ou com) dado útil.

Dica

Na lousa usamos uma *lista* ordenada para achar os dois menores. Na Aula 7 isso vira heap — mesmo serviço, mais barato. Não invente heap hoje se ainda não fechou a Aula 7: a lógica é a que importa.

Roteiro das 2h30

- ▶ 0:00–0:20 — o problema dos 8 bits e o prefixo-livre (figuras na apostila).
- ▶ 0:20–0:55 — **BANANA** na mão: contar, juntar, árvore, bits.
- ▶ 0:55–1:10 — descompactar bit a bit, sem pular.
- ▶ 1:10–2:00 — live coding (`exemplos/huffman.py`) e o arquivo `.huff`.
- ▶ 2:00–2:20 — o que o ZIP acrescenta (DEFLATE, uma ideia).
- ▶ 2:20–2:30 — prática: compactar **ABRA** na mão e conferir no código.

Atenção

Não pule o exemplo na mão. Quem só copia o `.py` não sabe dizer *por que* o `A` ganhou o bit `0`. Na prova e na apresentação, isso pesa mais que a sintaxe.

Compactação com Huffman

O problema (8 bits para tudo)

Um arquivo de texto em ASCII (e o `str` “cru” que você imagina na RAM) trata cada caractere como um bloco de 8 bits. A letra **A** e a letra **Z** custam *igual*, mesmo se o **A** aparecer mil vezes e o **Z** uma.

BANANA tem 6 letras. Em ASCII: $6 \times 8 = 48$ bits. Só que o **A** aparece 3 vezes, o **N** duas, o **B** uma. Se o frequente puder ser um bit só, o arquivo encolhe.

Mesmo texto, dois jeitos de gastar bits

ASCII: 8 bits para todo mundo		
B	01000010	
A	01000001	
N	01001110	
BANANA = 48 bits		

Huffman: o frequente paga menos		
A	0	aparece 3x
N	11	aparece 2x
B	10	aparece 1x
BANANA = 9 bits		

À esquerda, cada letra paga 8 bits. À direita, o A (o mais comum) paga 1 bit.

Resposta em uma frase

Huffman monta uma árvore em que letra frequente fica *perto da raiz* (código curto) e letra rara fica *longe* (código longo). Compactar é descer a árvore e anotar 0/1. Descompactar é ler os bits, descer, emitir a folha e voltar à raiz.

Dica

David Huffman inventou isso em 1952, ainda aluno. A pergunta do professor era: qual o melhor código se eu já conheço as frequências? A resposta é esta árvore.

Código prefixo-livre

Código curto só funciona se o descompressor souber *onde* um código acaba e o próximo começa. A regra: *nenhum código é prefixo de outro*. Isso se chama código prefixo-livre.

- ▶ Pode: **A=0**, **B=10**, **N=11**. O bit **0** sozinho já é A. **10** não começa com um A isolado no meio do caminho — você só emite quando chega na *folha*.
- ▶ Não pode: **A=0** e **B=00**. O primeiro **0** já seria A. O **B** nunca nasce.
- ▶ A árvore *garante* a regra: só emite na folha. Caminho interno nunca é letra.

Código prefixo-livre: nenhum código começa o outro

Pode: 0 10 11

Lendo 10011...

10 -> B

0 -> A

11 -> N

Nunca fica em dúvida onde corta.

Não pode: 0 e 00

A = 0 B = 00

Chegou o bit 0...

Ja e A? Ou espera o segundo 0 do B?

O descompressor trava. Por isso a árvore.

Se um código começa o outro, o leitor não sabe quando cortar. A árvore impede isso.

Atenção

Huffman *não* é BST. Na BST, esquerda/direita dependem de `valor < no.valor`. Aqui esquerda/direita são só os bits 0 e 1. A letra mora na folha.

Exemplo na mão: BANANA

Escreva B A N A N A no quadro. Peça para a turma contar *antes* de abrir o código. Se o número não bater, o resto da aula desanda.

Passo 1 contar BANANA

B A N A N A

Letra	Quantas vezes	Vira no da árvore
A	3	folha A, peso 3
N	2	folha N, peso 2
B	1	folha B, peso 1

A=3, N=2, B=1. Cada letra vira uma árvore de um nó só, com peso = frequência.

contar frequências

```
def contar(texto):
    freq = {}
    for letra in texto:
        freq[letra] = freq.get(letra, 0) + 1
    return freq
```

Passo 1. Três folhas: A(3), N(2), B(1). Isso é a *floresta*. Ainda não há pai.

Juntar sempre os dois menores

Enquanto houver mais de uma árvore na floresta: tire as duas de *menor peso*, crie um pai cujo peso é a soma, e devolva o pai. Empate de peso: ganha quem entrou primeiro (*ordem* menor). No código, *A* entra antes de *B* e *N* (ordem alfabética na montagem).

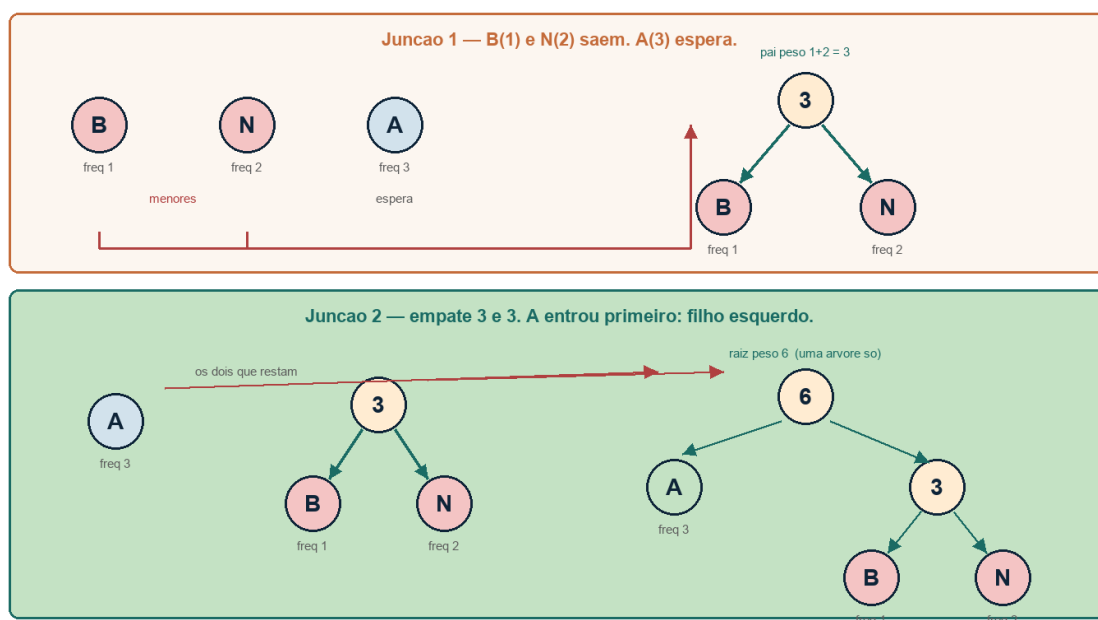
uma junção

```
floresta.sort(key=lambda no: (no.freq, no.ordem))
a = floresta.pop(0)          # o menor
b = floresta.pop(0)          # o segundo menor
pai = No(a.freq + b.freq, esquerda=a, direita=b)
floresta.append(pai)
```

Passo 2. Menores: *B*(1) e *N*(2). Pai peso 3, esquerda *B*, direita *N*. Floresta: *A*(3) e pai (3).

Passo 3. Empate 3 e 3. *A* tem *ordem* menor, sai primeiro, vira filho *esquerdo*. O pai *BN* vira filho direito. Sobra *uma* árvore, peso 6 — fim.

Passos 2 e 3 sempre junte os dois menores



Duas junções. Na segunda, A e o pai BN têm o mesmo peso; A entra à esquerda.

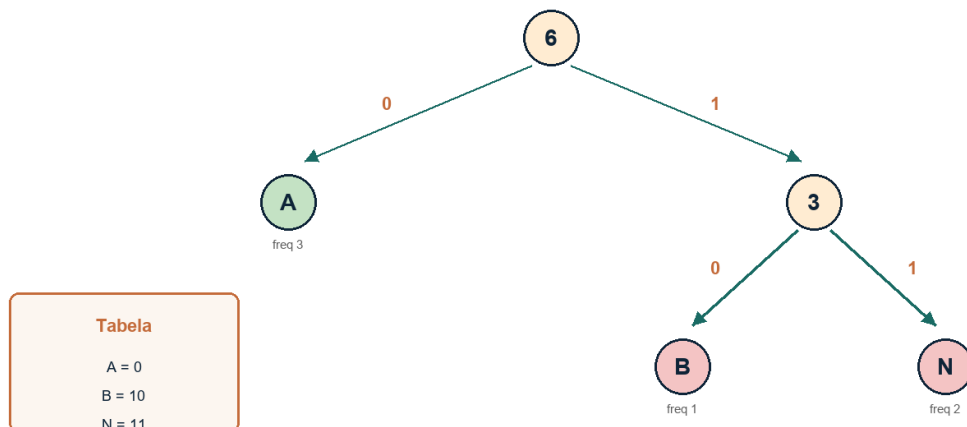
Dica

Se a turma juntar em outra ordem de empate, os *bits* mudam (*A=1* em vez de *A=0*), mas o tamanho continua 9 bits e o texto volta igual — desde que compactar e descompactar usem a *mesma* árvore. Por isso o arquivo `.huff` guarda a tabela.

Ler os bits na árvore

Raiz até a folha: cada passo à esquerda anota *0*, à direita anota *1*. O código da letra é o caminho.

Passo 4 esquerda = 0 direita = 1



A está logo à esquerda da raiz (código 0). B = 10, N = 11.

- ▶ A: esquerda → 0.
- ▶ B: direita, esquerda → 10.
- ▶ N: direita, direita → 11.

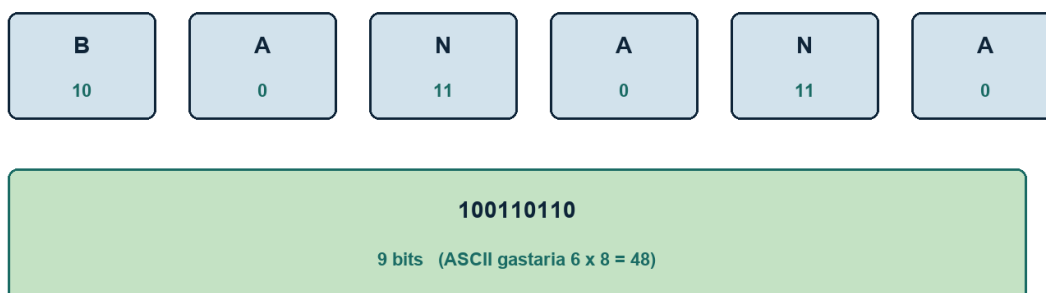
Compactar letra a letra

Troque cada letra pelo código e concatene. Não coloque espaço no arquivo real; no quadro o espaço só ajuda a enxergar.

montar a bitstring

```
codigos = {"A": "0", "B": "10", "N": "11"}
bits = "".join(codigos[letra] for letra in "BANANA")
# 10 + 0 + 11 + 0 + 11 + 0 -> 100110110
```

Compactar BANANA letra a letra



B=10, A=0, N=11, A=0, N=11, A=0 → 100110110. De 48 bits para 9.

Contas para o quadro: $3 \times 1 + 2 \times 2 + 1 \times 2 = 3 + 4 + 2 = 9$ bits. Isso é a soma **frequência** × **comprimento do código**. Huffman escolhe os comprimentos para essa soma ficar pequena (ótimo entre os códigos prefixo-livres, quando as frequências são as da mensagem).

Descompactar bit a bit

Comece na raiz. Leia *um* bit. 0 desce à esquerda, 1 à direita. Chegou em folha? Escreva a letra e volte à raiz. Não pule bits.

Descompactar: um bit, um passo na árvore. Folha? Emita e volte a raiz.



Depois: 0 -> A, 11 -> N, 0 -> A. Texto: BANANA

Os dois primeiros bits 10 não são “dez”: são um passo (1) e outro (0) até a folha B.

descer até a folha

```
no = raiz
for bit in bits:
    no = no.esquerda if bit == "0" else no.direita
    if no. eh_folha():
        print(no.simbolo, end="")
        no = raiz
```

Passo 1. 1 → direita (nó interno). 0 → esquerda → folha B. Volta à raiz.

Passo 2. 0 → esquerda → folha A.

Passo 3. 1 → direita (interno). 1 → direita → folha N.

Passo 4. 0 → A. 11 → N. 0 → A. Texto: BANANA.

Atenção

Se no fim dos bits você *não* estiver de novo na raiz, a bitstring está quebrada ou a tabela não é a mesma da compactação. Não “chute” a última letra.

Live coding

Abra `exemplos/huffman.py`. Digite com a turma nesta ordem: No → contar → construir_arvore (o while das junções) → tabela_codigos → compactar / descompactar. Só então o arquivo `.huff`.

`exemplos/huffman.py`

```
"""Compactação Huffman – aula extra (hoje).
```

```
Texto de teste: BANANA
```

```

A aparece 3 vezes, N 2, B 1.
Códigos: A=0, B=10, N=11
Bits: 100110110 (9 bits em vez de 48 do ASCII)
"""

from pathlib import Path

class No:
    def __init__(self, freq, simbolo=None, esquerda=None, direita=None, ordem=0):
        self.freq = freq
        self.simbolo = simbolo
        self.esquerda = esquerda
        self.direita = direita
        self.ordem = ordem

    def eh_folha(self):
        return self.simbolo is not None

def contar(texto):
    freq = {}
    for letra in texto:
        freq[letra] = freq.get(letra, 0) + 1
    return freq

def construir_arvore(freq):
    floresta = []
    ordem = 0
    for simbolo in sorted(freq):
        floresta.append(No(freq[simbolo], simbolo, ordem=ordem))
        ordem += 1

    if len(floresta) == 1:
        unico = floresta[0]
        return No(unico.freq, esquerda=unico, ordem=ordem)

    while len(floresta) > 1:
        floresta.sort(key=lambda no: (no.freq, no.ordem))
        a = floresta.pop(0)
        b = floresta.pop(0)
        pai = No(a.freq + b.freq, esquerda=a, direita=b, ordem=ordem)
        ordem += 1
        floresta.append(pai)
    return floresta[0]

def tabela_codigos(raiz):
    codigos = {}

    def caminhar(no, prefixo):
        if no.eh_folha():

```

```

        codigos[no.simbolo] = prefixo or "0"
        return
    caminhar(no.esquerda, prefixo + "0")
    caminhar(no.direita, prefixo + "1")

    caminhar(raiz, "")
    return codigos

def compactar(texto):
    freq = contar(texto)
    raiz = construir_arvore(freq)
    codigos = tabela_codigos(raiz)
    bits = "".join(codigos[letra] for letra in texto)
    return raiz, codigos, bits

def descompactar(raiz, bits):
    letras = []
    no = raiz
    for bit in bits:
        no = no.esquerda if bit == "0" else no.direita
        if no.eh_folha():
            letras.append(no.simbolo)
            no = raiz
    return "".join(letras)

def salvar_huff(caminho, codigos, bits):
    with open(caminho, "w", encoding="utf-8") as arq:
        arq.write("HUFFMAN\n")
        for simbolo in sorted(codigos):
            arq.write(f"{simbolo} {codigos[simbolo]}\n")
        arq.write("---\n")
        arq.write(bits)
        arq.write("\n")

def abrir_huff(caminho):
    with open(caminho, "r", encoding="utf-8") as arq:
        linhas = [linha.rstrip("\n") for linha in arq]
    if not linhas or linhas[0] != "HUFFMAN":
        raise ValueError("Arquivo .huff inválido.")
    codigos = {}
    i = 1
    while i < len(linhas) and linhas[i] != "---":
        simbolo, codigo = linhas[i].split(" ", 1)
        codigos[simbolo] = codigo
        i += 1
    bits = "".join(linhas[i + 1 :])
    return codigos, bits

def descompactar_tabela(codigos, bits):

```

```

inverso = {codigo: simbolo for simbolo, codigo in codigos.items()}
letras = []
atual = ""
for bit in bits:
    atual += bit
    if atual in inverso:
        letras.append(inverso[atual])
        atual = ""
if atual:
    raise ValueError("Sobrou bit sem código. A tabela ou os bits estão errados.")
return "".join(letras)

if __name__ == "__main__":
    texto = "BANANA"
    raiz, codigos, bits = compactar(texto)
    print("Frequência:", contar(texto))
    print("Códigos:", codigos)
    print("Bits:", bits)
    print("Tamanho ASCII:", len(texto) * 8, "bits")
    print("Tamanho Huffman:", len(bits), "bits")
    volta = descompactar(raiz, bits)
    print("Volta pela árvore:", volta)
    saida = Path(__file__).resolve().parent / "banana.huff"
    salvar_huff(saida, codigos, bits)
    tab, bits_arquivo = abrir_huff(saida)
    print("Volta pelo arquivo:", descompactar_tabela(tab, bits_arquivo))

```

Boa prática

O `ordem` no nó é o desempate. Sem ele, dois pais com o mesmo peso deixam o `sort` instável entre máquinas — e a tabela muda. Frequência primeiro, ordem depois.

O arquivo .huff

Na aula o arquivo é *texto*, de propósito. Você abre no VS Code e vê a tabela. Um compactador “de verdade” empacota os bits em bytes (`int(bits, 2)` de 8 em 8) e guarda um cabeçalho binário. Isso é o dever de casa, não o quadro das 19h.

- ▶ Primeira linha: **HUFFMAN** (assinatura).
- ▶ Depois: **letra código**, uma por linha.
- ▶ Uma linha `---` separa a tabela dos bits.
- ▶ Por último: a bitstring (`100110110`).

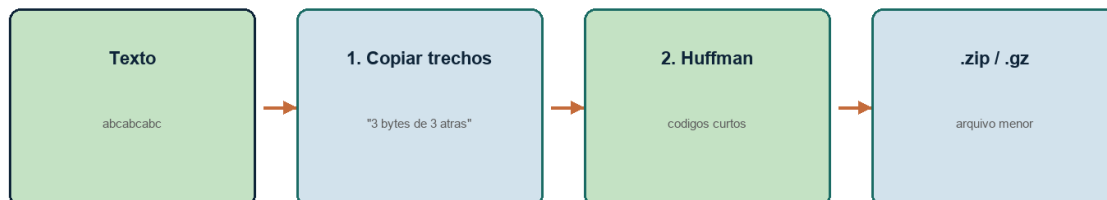
Dica

Tem dois jeitos de voltar o texto: descer a árvore (`descompactar`) ou ir acumulando bits até achar um código na tabela invertida (`descompactar_tabela`). Os dois precisam da *mesma* tabela que foi salva.

O que o ZIP faz a mais

Huffman trata *símbolo por símbolo*. Ele não percebe que `abcabcabc` é o mesmo bloco três vezes. O DEFLATE (o motor do gzip e de muitos `.zip`) faz duas etapas:

ZIP / gzip (DEFLATE): duas ideias na fila



Primeiro copia trechos que já apareceram (LZ77). Depois aplica Huffman nesses códigos.

- ▶ LZ77: “copie 3 bytes de 3 posições atrás” em vez de repetir `abc`.
- ▶ Huffman: os códigos de “cópia” e de “literal” que mais aparecem ficam curtos.
- ▶ JPEG e MP3 já vieram comprimidos. Jogar no ZIP quase não encolhe — às vezes até cresce, por causa do cabeçalho.

Hoje vocês implementam só a segunda ideia (Huffman). A primeira (janela de cópia) é um projeto de extensão: uma fila/buffer do texto recente, no espírito da Aula 3.

Prática de hoje

Na mão (10 min) — ABRA

Conte A, B, R. Monte a floresta, faça as junções, escreva a tabela e a bitstring. Depois rode o script trocando `texto = "ABRA"` e compare. Os bits podem diferir se o empate for outro; o tamanho e a volta do texto é que precisam bater.

No código (10 min)

Compacte uma frase com o nome da turma. Confira se `descompactar` devolve a frase. Abra o `.huff` no editor: a tabela tem de fazer sentido com o que está no quadro.

Antes da próxima aula, você precisa conseguir

- ▶ Explicar, sem olhar o código, por que `A` em `BANANA` fica com 1 bit.
- ▶ Descompactar `100110110` na árvore da apostila, bit a bit.
- ▶ Dizer o que é prefixo-livre — e dar um contraexemplo (`0` e `00`).
- ▶ Rodar `python exemplos/huffman.py` e ver `BANANA` voltar.
- ▶ Uma frase: Huffman não é BST; ZIP não é só Huffman.